PDK 2.HAFTA Notları

Arda Kumartaşlıoğlu

22-09-2025

1 Bölüm Konuları

- Programlama Dilleri Kavramlarını İncelemenin Nedenleri
- Programlama Alanları
- Dil Değerlendirme Kriterleri
- Dil Tasarımını Etkileyen Faktörler
- Dil Kategorileri
- Dil Tasarımında Değiş-Tokuşlar
- Uygulama Yöntemleri
- Programlama Ortamları

1.1 Programlama Dilleri Kavramlarını İncelemenin Nedenleri

- Fikirleri ifade etme becerisinin artması
- Uygun dilleri seçmek için geliştirilmiş arka plan
- Yeni dilleri öğrenme becerisinde artış
- Zaten bilinen dilleri daha iyi kullanmak
- Bilgisayar teknolojisinin temel gelişimini anlamak

1.2 Programlama Alanları

1.2.1 Bilimsel Uygulamalar

Fortran

1.2.2 İşletmelere yönelik uygulamalar

COBOL

1.2.3 Yapay zeka

LISP

1.2.4 Sistem programlama

C

1.2.5 Web Yazılımı

HTML,PHP,Java

1.3 Dil Değerlendirme Kriterleri

- **Okunabilirlik (Readability)**: Programların okunması ve anlaşılmasının kolaylığı.
- **Yazılabilirlik (Writability)**: Bir dilin program oluşturmak için kullanılabilme kolaylığı.
- **Güvenilirlik (Reliability)**: Yazdığınız programın beklendiği gibi sorunsuz çalışması.
- **Maliyet (Cost)**: Bir programın ne kadar masrafa yol açtığıdır.
- **Diğerleri (Others)**
  - Taşınabilirlik: Yazdığınız kodun farklı bilgisayarlarda veya işletim sistemlerinde sorunsuz çalışabilmesidir. Örneğin, Java bu konuda oldukça başarılıdır.
  - Genellik: Dilin ne kadar geniş bir alanda kullanılabilir olduğudur. Bazı diller sadece belirli bir işe (örneğin web sitesi yapmaya) odaklanmışken, bazıları (örneğin Python) hemen hemen her alanda kullanılabilir.
  - İyi Tanımlanmışlık: Dilin kurallarının net ve açık olmasıdır. Bu sayede, kodun nasıl çalışacağı konusunda kafa karışıklığı yaşanmaz.

1.3.1 Dil Değerlendirme Kriteri: Okunabilirlik (Readability)

- **Genel Basitlik (Overall Simplicity)**
  - Yönetilebilir bir dizi özellik ve yapı.
  - Minimal özellik çokluğu.
  - Minimal operatör aşırı yükleme.
- **Ortogonalite (Orthogonality)**
  - Nispeten küçük bir dizi ilkel yapı, nispeten az sayıda şekilde birleştirilebilir.

  - Her türlü kombinasyon geçerlidir.
- **Veri Türleri (Data Types)**
  - Uygun şekilde önceden tanımlanmış veri türleri.
- **Söz Dizimi Hususları (Syntax Considerations)**
  - Tanımlayıcı formlar: esnek bileşim.
  - Özel kelimeler ve bileşik cümleler kurma yöntemleri.
  - Biçim ve anlam: kendini tanımlayan yapılar, anlamlı anahtar kelimeler.

1.3.2 Dil Değerlendirme Kriteri: Yazılabilirlik (Writability)

- **Basitlik ve Ortogonalite**
  - Az sayıda yapı, az sayıda temel öğe, bunları birleştirmek için az sayıda kural.
- **Soyutlama (Abstraction) Desteği**
  - Ayrıntıları göz ardı etmeye izin verecek şekilde karmaşık yapıları veya işlemleri tanımlama ve kullanma becerisi.
- **İfade Gücü (Expressivity)**
  - İşlemleri belirtmek için nispeten kullanışlı bir dizi yöntem.
  - Operatörlerin gücü ve sayısı ile önceden tanımlanmış fonksiyonlar.

1.3.3 Dil Değerlendirme Kriteri: Güvenilirlik (Reliability)

- **Tip Kontrolü (Type Checking)**
  - Tip hataları için test etme.
- **İstisna İşleme (Exception Handling)**
  - Çalışma zamanı hatalarını yakalayın ve düzeltici önlemler almak.
- **Örtüşme (Aliasing)**
  - Aynı bellek konumu için iki veya daha fazla farklı referanslama yönteminin varlığı.
- **Okunabilirlik ve Yazılabilirlik (Readability and Writability)**
  - Bir algoritmayı "doğal" yollarla ifade etmeyi desteklemeyen bir dil, "doğal olmayan" yaklaşımların kullanılmasını gerektirecek ve bu da güvenilirliği azaltacaktır.

1.3.4 Dil Değerlendirme Kriteri: Maliyet (Cost)

- Programcıları dil kullanımı konusunda eğitmek.
- Program yazma (belirli uygulamalara yakınlık).
- Programları derleme (compiling).
- Programları çalıştırma (executing).
- Dil uygulama sistemi: Ücretsiz derleyicilerin kullanılabilirliği.
- Güvenilirlik: Düşük güvenilirlik yüksek maliyetlere yol açar.
- Programların sürdürülmesi.

1.3.5 Dil Değerlendirme Kriteri: Diğerleri (Others)

- **Taşınabilirlik (Portability)**
  - Programların bir uygulamadan diğerine kolaylıkla taşınabilmesi.
- **Genellik (Generality)**
  - Geniş bir uygulama yelpazesine uygulanabilirlik.
- **İyi Tanımlanmışlık (Well-Definedness)**
  - Dilin resmi tanımının eksiksizliği ve kesinliği.

1.4 Dil Tasarımını Etkileyen Faktörler

- **Bilgisayar Mimarisi**
  - Diller, von Neumann mimarisi olarak bilinen yaygın bilgisayar mimarisi etrafında geliştirilmiştir.
- **Program Tasarım Metodolojileri**
  - Yeni yazılım geliştirme metodolojileri (örneğin, nesne yönelimli yazılım geliştirme), yeni programlama paradigmalarına ve dolayısıyla yeni programlama dillerine yol açmıştır.

Von Neumann Mimarisi

- **Getir-yürüt-döngüsü (Fetch-execute-cycle)** (von Neumann mimarisine sahip bir bilgisayarda)
  - Program sayacını başlat
  - Sonsuza kadar tekrarla
  - Sayacın işaret ettiği komutu al

  - Sayacı bir artır
  - Komutu çöz
  - Komutu yürüt
  - Döngüyü sonlandır

—

Programlama Metodolojilerinin Etkileri

- **1950'ler ve 1960'ların başı:** Basit uygulamalar; makine verimliliği konusunda endişe.
- **1960'ların sonu:** İnsan verimliliği önem kazandı; okunabilirlik, daha iyi kontrol yapıları.
  - Yapısal programlama (Structured programming).
  - Yukarıdan aşağıya (Top-down) tasarım ve adım adım (step-wise) iyileştirme.
- **1970'lerin sonu:** Süreç odaklıdan veri odaklıya geçiş.
  - Veri soyutlama (abstraction).
- **1980'lerin ortası:** Nesne yönelimli programlama.
  - Veri soyutlama (abstraction) + kalıtım + polimorfizm.

—

1.5 Dil Kategorileri

- **Zorunlu (Imperative)**
  - Temel özellikler değişkenler, atama ifadeleri ve iterasyondur.
  - Nesne yönelimli programlamayı destekleyen diller dahildir.
  - Scripting dilleri dahildir.
  - Görsel diller dahildir.
  - Örneğin: C, Java, Perl, JavaScript, Visual BASIC .NET, C++.
- **İşlevsel (Functional)**
  - Hesaplamaların yapılmasının ana yolu, verilen parametrelere fonksiyonlar uygulamaktır.
  - Örneğin: LISP, Scheme, ML, F#.
- **Mantık (Logic)**

  - Kural tabanlı (kurallar belirli bir sıraya göre belirtilmemiştir).
  - Örneğin: Prolog.
- **İşaretleme/programlama melezi (Markup/programming hybrid)**
  - Bazı programlamaları desteklemek için genişletilmiş işaretleme (markup) dilleri.
  - Örneğin: JSTL, XSLT.

—

1.6 Dil Tasarımında Değiş-Tokuşlar

- **Güvenilirlik ve uygulama maliyeti (Reliability vs. cost of execution)**
  - Örneğin: Java, dizi öğelerine yapılan tüm referansların doğru indeksleme açısından kontrol edilmesini gerektirir, bu da yürütme maliyetlerinin artmasına neden olur.
- **Okunabilirlik ve yazılabilirlik (Readability vs. writability)**
  - Örneğin: APL, karmaşık hesaplamaların kompakt bir programda yazılmasını sağlayan birçok güçlü operatör (ve çok sayıda yeni sembol) sunar, ancak bunun bedeli okunabilirliğin düşmesidir.
- **Yazılabilirlik (esneklik) ve güvenilirlik (Writability (flexibility) vs. reliability)**
  - Örneğin: C++ işaretçileri (pointers) güçlü ve çok esnektir, ancak güvenilir değildir.

—

1.7 Uygulama (Implementation) Yöntemleri

- **Derleme (Compilation)**
  - Programlar makine diline çevrilir; JIT sistemleri içerir.
  - Kullanım: Büyük ticari uygulamalar.
- **Saf Yorumlama (Pure Interpretation)**
  - Programlar, yorumlayıcı (interpreter) olarak bilinen başka bir program tarafından yorumlanır.
  - Kullanım: Küçük programlar veya verimlilik önemli değilse.
- **Hibrit Uygulama Sistemleri (Hybrid Implementation Systems)**
  - Derleyiciler ve saf yorumlayıcılar arasında bir uzlaşma.
  - Kullanım: Verimlilik öncelikli değilse küçük ve orta ölçekli sistemler.

1.7.1 Derleme (Compilation)

- Yüksek seviyeli programı (kaynak dil) makine koduna (makine dili) çevirir.
- Yavaş çeviri, hızlı uygulama.
- Derleme süreci birkaç aşamadan oluşur:
  - **Sözcük analizi (Lexical Analysis)**: Kaynak programdaki karakterleri sözcük birimlerine dönüştürür.
  - **Söz Dizimi analizi (Syntax Analysis)**: Sözcük birimlerini, programın söz dizimsel yapısını temsil eden ayrıştırma ağaçlarına dönüştürür.
  - **Anlamsal Analiz (Semantics Analysis)**: Ara kod oluşturma.
  - **Kod Üretimi (Code Generation)**: Makine kodu üretilir.
- **Ek Derleme Terminolojileri**
  - **Yükleme modülü (Load module)**: Kullanıcı ve sistem kodu birlikte.
  - **Bağlama ve yükleme (Linking and loading)**: Sistem program birimlerini toplama ve bunları bir kullanıcı programına bağlama işlemi.

—

Von Neumann Dar Boğazı (Bottleneck)

- Bir bilgisayarın belleği ile işlemcisi arasındaki bağlantı hızı, bilgisayarın hızını belirler.
- Program talimatları genellikle bağlantı hızından çok daha hızlı bir şekilde yürütülebilir; bu nedenle bağlantı hızı bir dar boğaz oluşturur.
- Von Neumann Dar Boğazı olarak bilinir; bilgisayarların hızındaki başlıca sınırlayıcı faktördür.

—

1.7.2 Saf Yorumlama (Pure Interpretation)

- Çeviri yok.
- Programların daha kolay uygulanması (çalışma zamanı hataları kolayca ve anında görüntülenebilir).
- Daha yavaş çalışma (derlenmiş programlara göre 10 ila 100 kat daha yavaş).
- Genellikle daha fazla alan gerektirir.
- Geleneksel yüksek seviyeli dillerde artık nadir görülen bir durumdur.

- Bazı web betik dillerinde (Web scripting languages) (örneğin JavaScript, PHP) önemli bir geri dönüş kazanmıştır.

—

1.7.3 Hibrit Uygulama (Implementation) Sistemleri

- Derleyiciler ve saf yorumlayıcılar arasında bir uzlaşma.
- Yüksek seviyeli bir dil programı, kolay yorumlamaya olanak tanıyan bir ara dile çevrilir.
- Saf yorumlamadan daha hızlı.
- Örneğin:
  - Perl programları, yorumlanmadan önce hataları tespit etmek için kısmen derlenir.
  - Java'nın ilk uygulamaları hibritti; ara form olan byte kodu, byte kodu yorumlayıcısı ve çalışma zamanı sistemi (bunların birlikte Java Sanal Makinesi olarak adlandırılır) bulunan herhangi bir makineye taşınabilirlik sağlar.

—

1.8 Programlama Ortamları

- Yazılım geliştirmede kullanılan araçların bir koleksiyonu.
- **UNIX**
  - Eski bir işletim sistemi ve araç koleksiyonu.
  - Günümüzde genellikle UNIX üzerinde çalışan bir GUI (örneğin CDE, KDE veya GNOME) aracılığıyla kullanılır.
- **Microsoft Visual Studio .NET**
  - Geniş ve karmaşık bir görsel ortam.
  - Herhangi bir .NET dilinde Web uygulamaları ve Web dışı uygulamalar oluşturmak için kullanılır.
- **NetBeans**
  - Java uygulamaları ile ilgili.

—

Özet

- Programlama dillerinin incelenmesi birçok nedenden dolayı değerlidir:
  - Farklı yapıları kullanma kapasitemizi artırmak.
  - Daha akıllı bir şekilde dil seçmemizi sağlar.
  - Yeni dilleri öğrenmeyi kolaylaştırır.
- Programlama dillerini değerlendirmek için en önemli kriterler şunlardır:
  - Okunabilirlik, yazılabilirlik, güvenilirlik, maliyet.
- Dil tasarımında en büyük etkenler makine mimarisi ve yazılım geliştirme metodolojileri olmuştur.
- Programlama dillerini uygulamaya koymanın başlıca yöntemleri şunlardır: derleme, saf yorumlama ve hibrit uygulama.